ARTIFICIAL INTELLIGENCE

CSE 3201

Informed Search (II)

Iterative-Deepening A* (IDA*)

- A* may require exponential storage

  Solution: *Iterative-deepening A* (IDA*)*

  – Just like Iterative Deepening, but…

  – Instead of using the actual depth so far to cut off searching,

    use f(N) (estimated *total* depth)

  – Explores paths within a given contour (f-cost limit) in a depth-first manner

- IDA* gives the same results as A*

- Since IDA* is basically a depth-first search, storage requirements are linear in length of path.

IDA* Search

- Just as iterative deepening solved the space problem of breadth-first search, iterative deepening A* (IDA*) eliminates the memory constraints of A* search algorithm without sacrificing solution optimality.
- Each iteration of the algorithm is a depth-first search that keeps track of the cost, f(n) = g(n) + h(n), of each node generated.
- As soon as a node is generated whose cost exceeds a threshold for that iteration, its path is cut off, and the search backtracks before continuing. The cost threshold is initialized to the heuristic estimate of the initial state, and in each successive iteration is increased to the total cost of the lowest-cost node that was pruned during the previous iteration.
- The algorithm terminates when a goal state is reached whose total cost does not exceed the current threshold.
- Since Iterative Deepening A* performs a series of depth-first searches, its memory requirement is linear with respect to the maximum search depth. In addition, if the heuristic function is admissible, IDA* finds an optimal solution.

Iterative Deepening A* (IDA*)

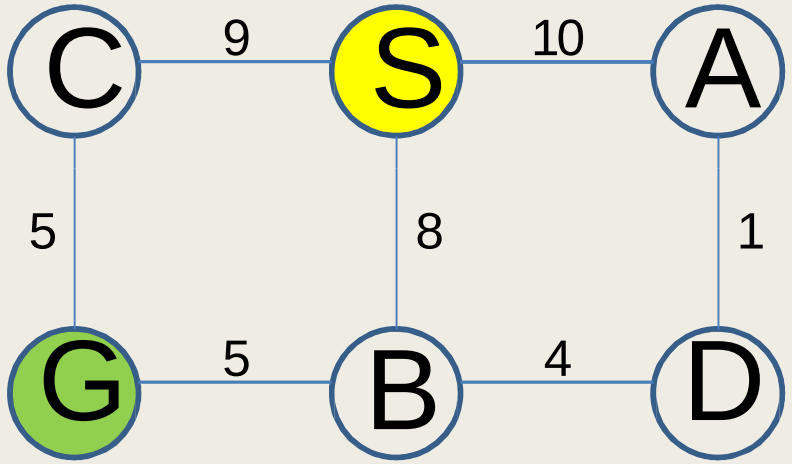
- How does it work:
  - *A cost threshold is set.*
  - *f(n) = g(n) + h(n) is computed in each iteration.*
  - *If f(n) < threshold we expand the node.*
  - *Else the branch is pruned (we don't expand it).*

  .....

  - *If a goal node is reached with lower or equal value than threshold then it is returned.*
  - *Else if a whole iteration has ended without reaching the goal, then another iteration is begun with a greater cost threshold.*
  - *The new cost threshold is set to the minimum cost of al nodes that were pruned on the previous iteration.*
  - *The cost Threshold for the first Iteration is set to the cost of the initial state.*

Exercise

- Perform the IDA* Algorithm on the following figure.

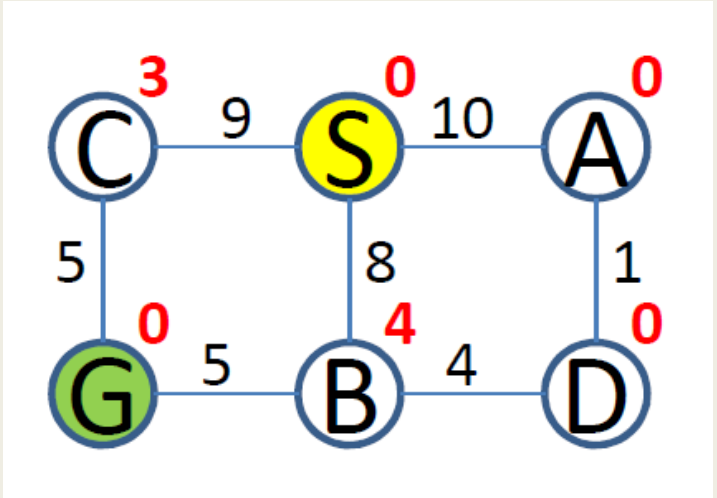


	S	A	B	C	D	G
heuristic	0	0	4	3	0	0

IDA* Search

$f(n) = g(n) + h(n)$

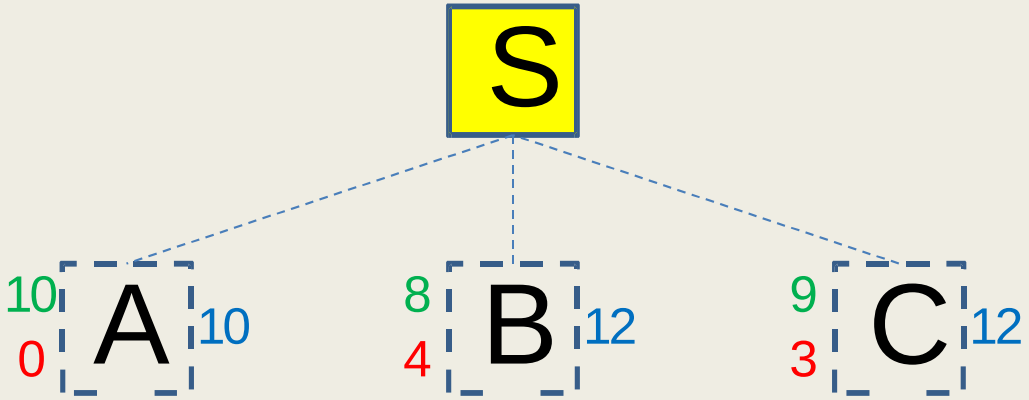




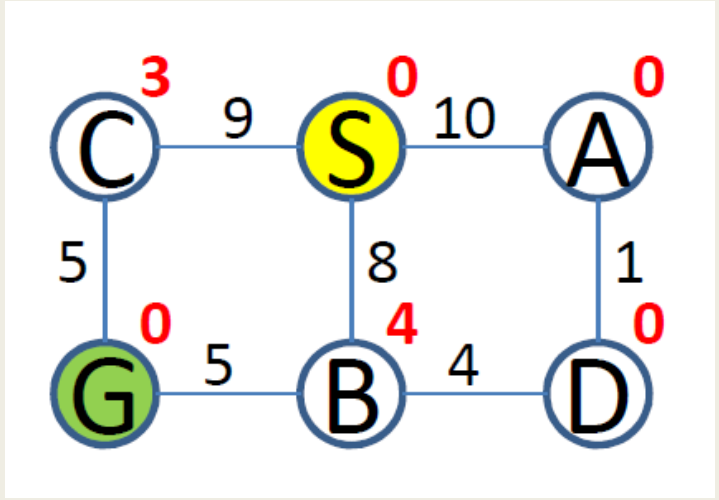
f-bound =0

f-new = ∞

IDA* Search



Children are explored depth-first!

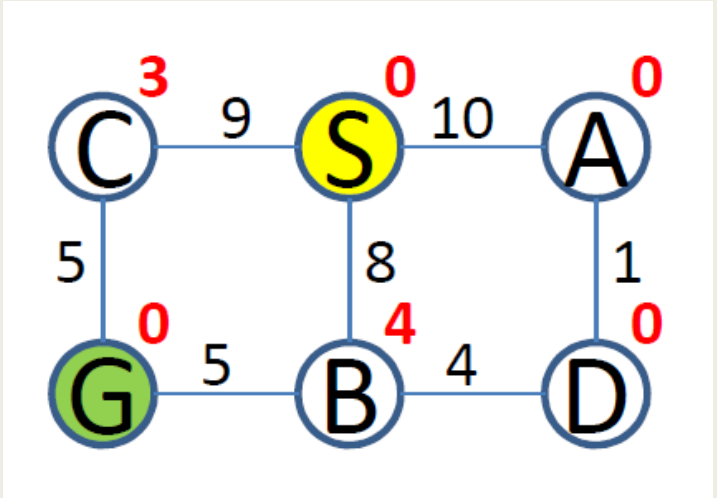


f-bound = 0

f-new = 10

IDA* Search

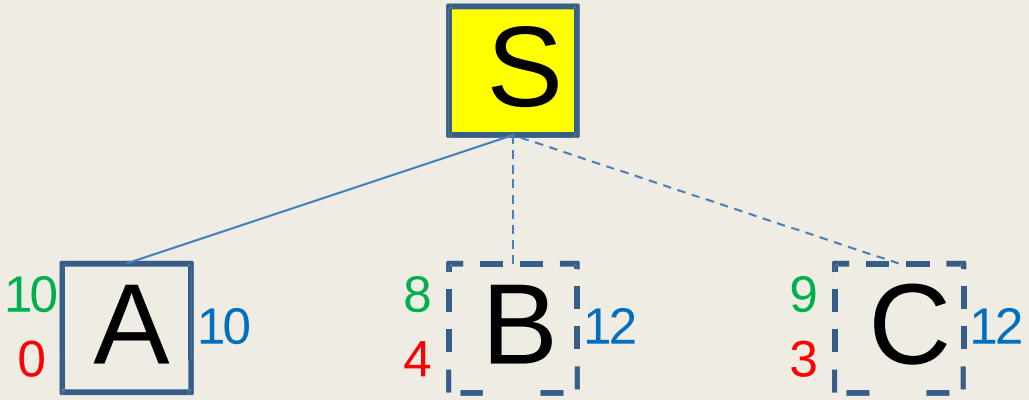


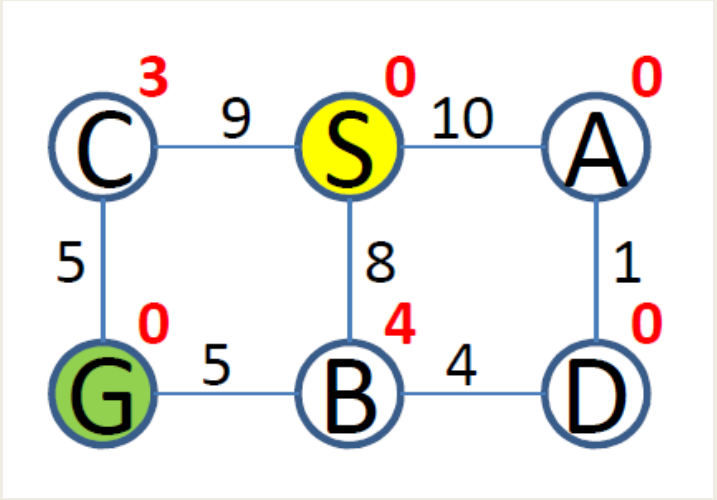


f-bound =10

f-new =∞

IDA* Search

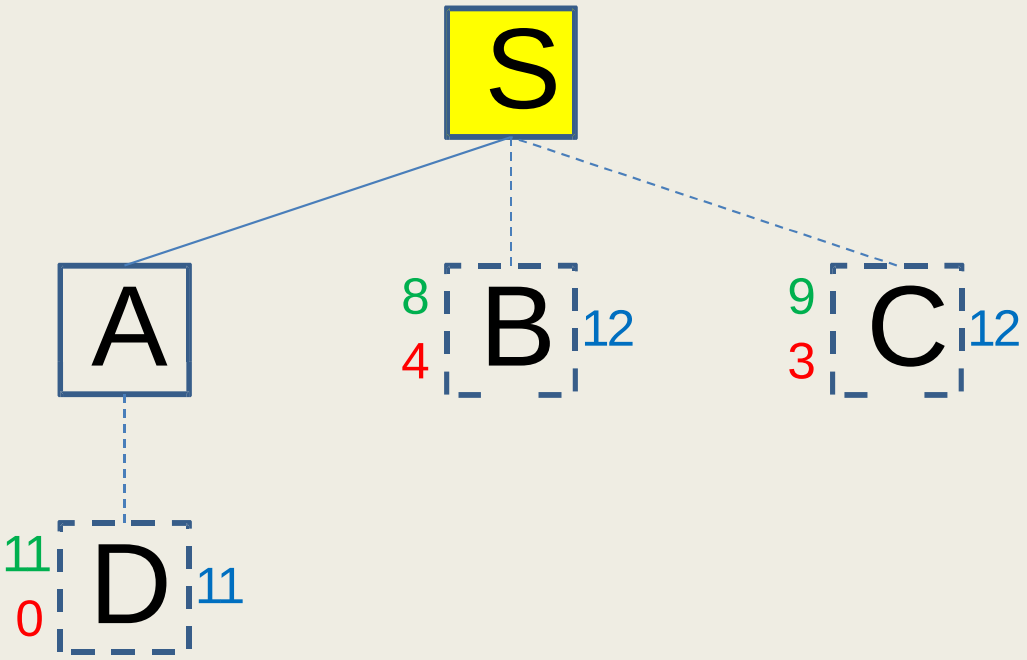


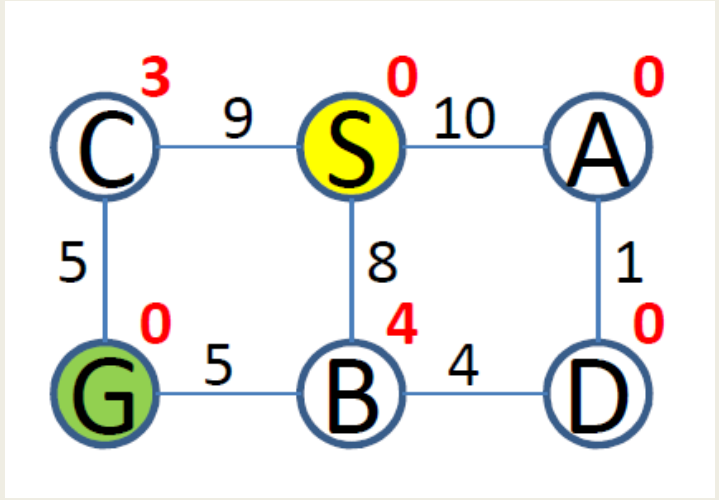


f-bound = 10

f-new = 12

IDA* Search



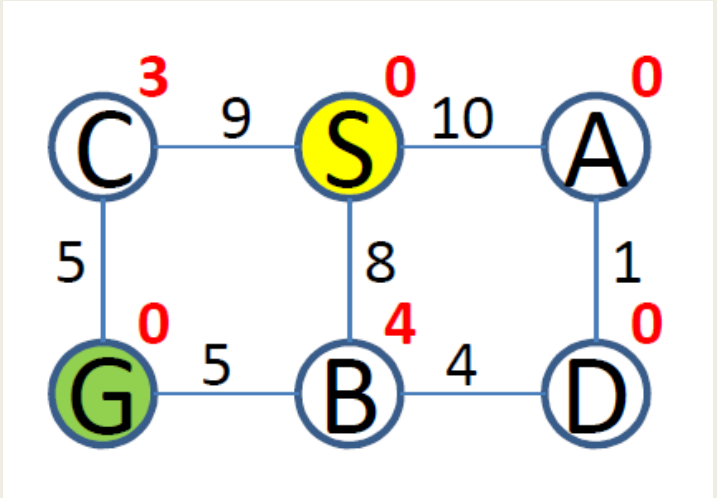


f-bound = 10

f-new = 11

IDA* Search

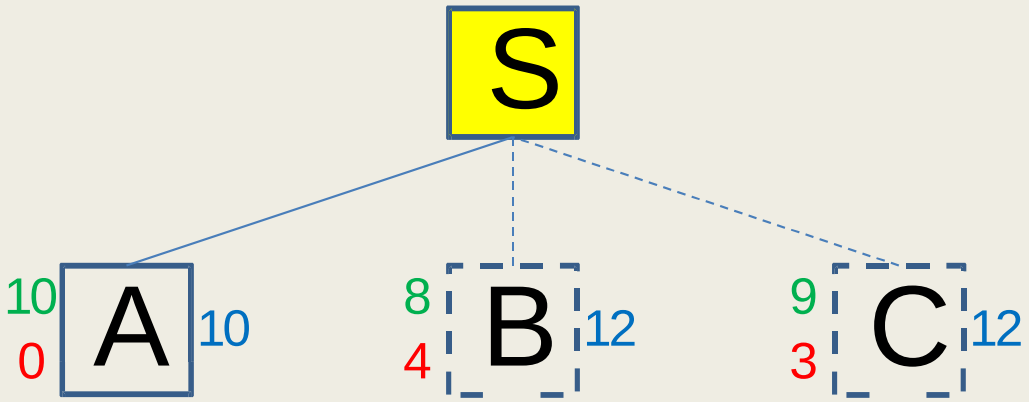


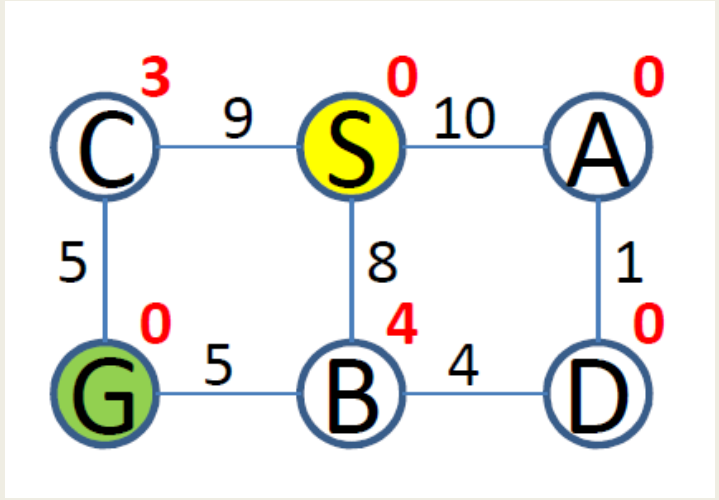


f-bound =11

f-new =∞

IDA* Search

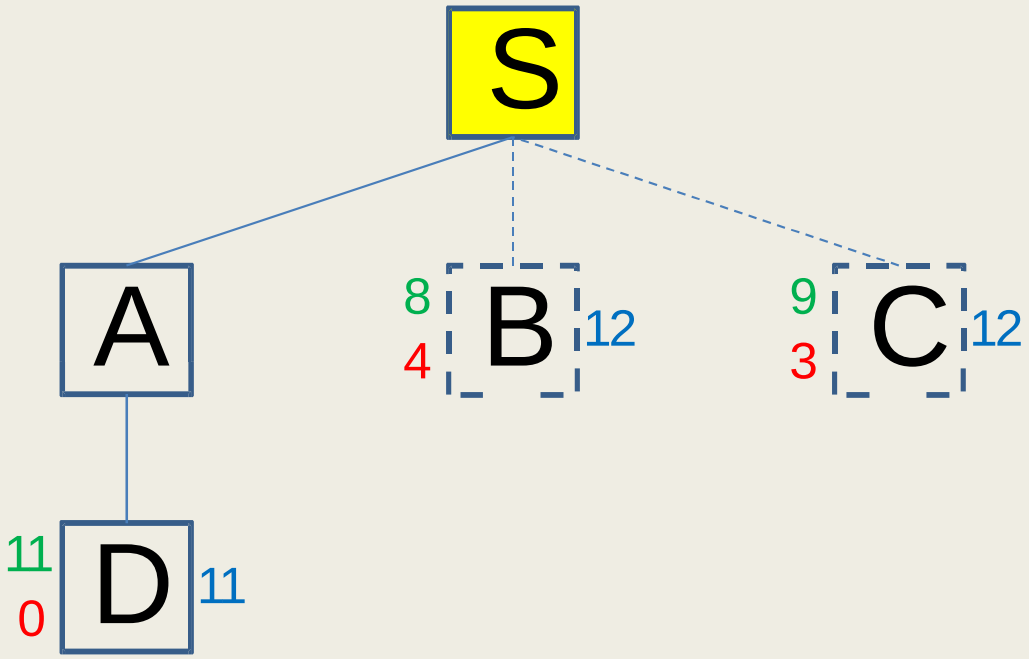


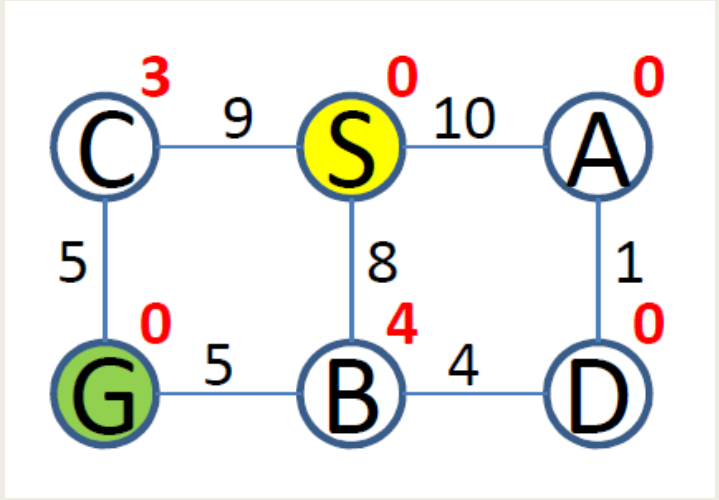


f-bound = 11

f-new = 12

IDA* Search

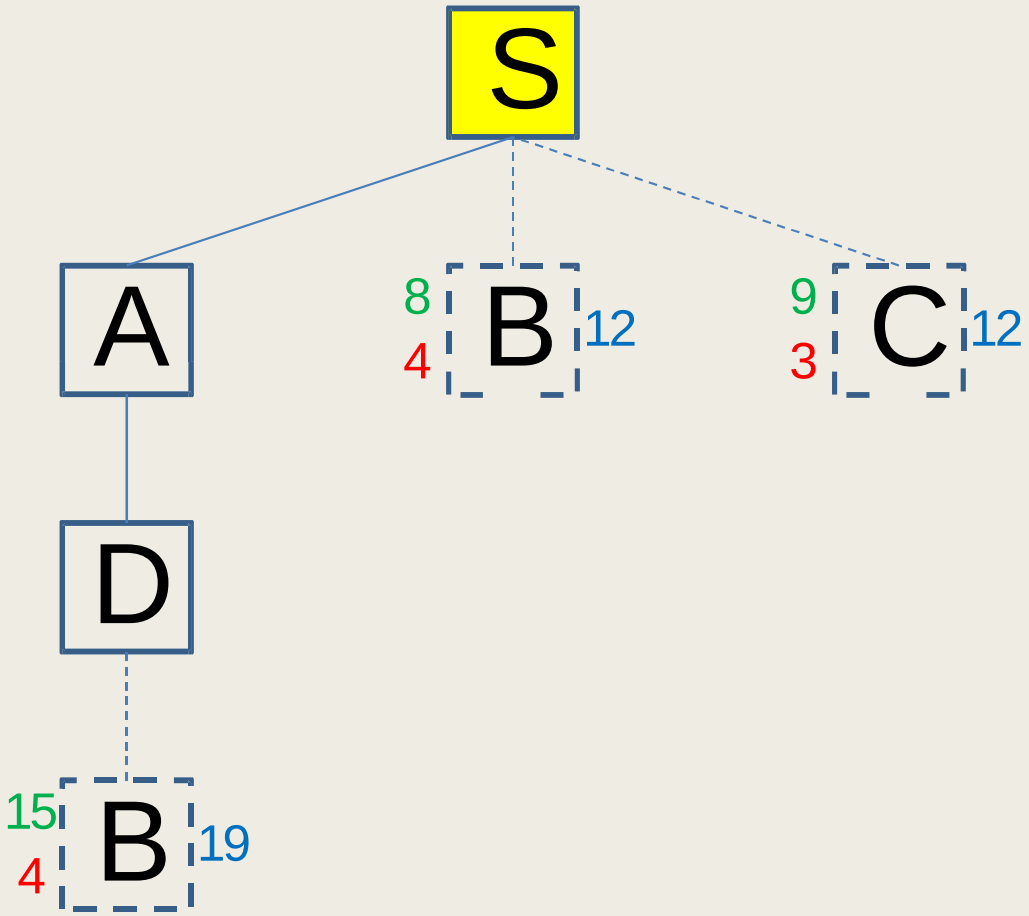


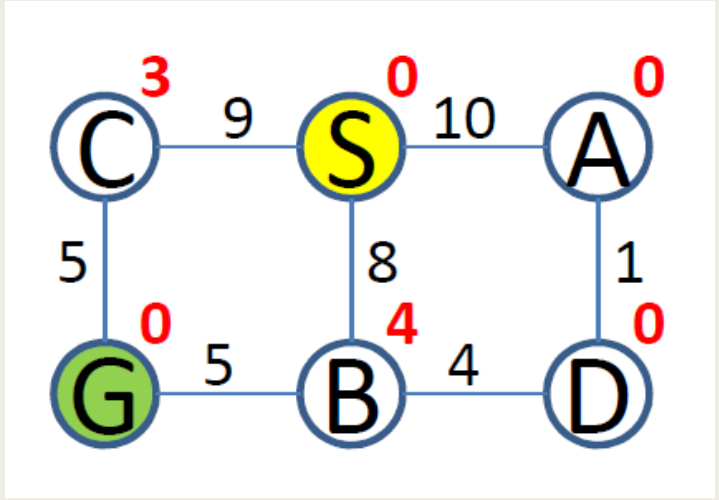


f-bound = 11

f-new = 12

IDA* Search



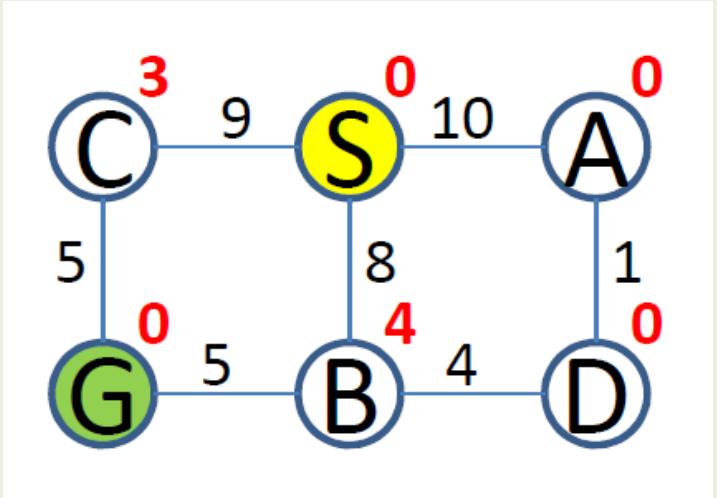


f-bound = 11

f-new = 12

IDA* Search

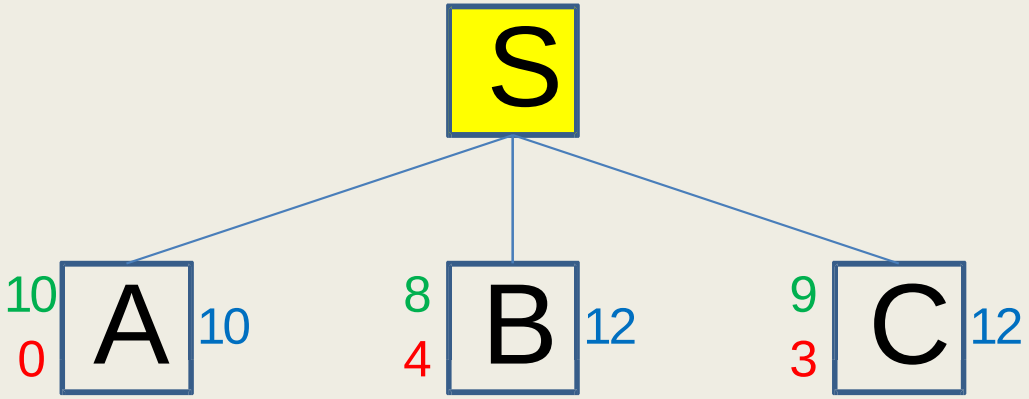


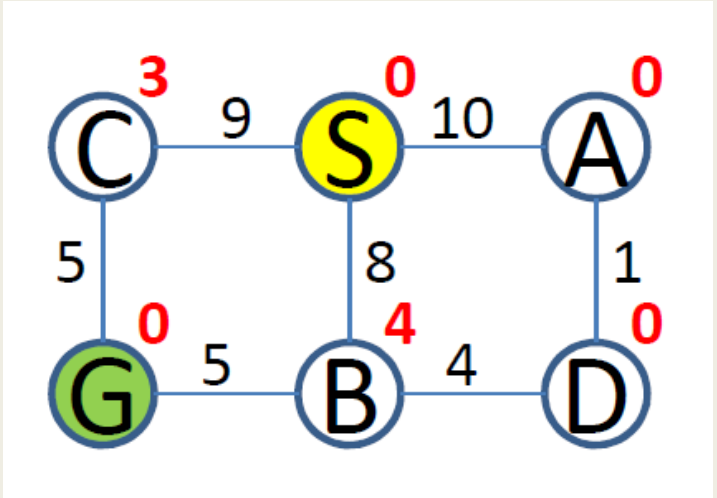


f-bound =12

f-new =∞

IDA* Search

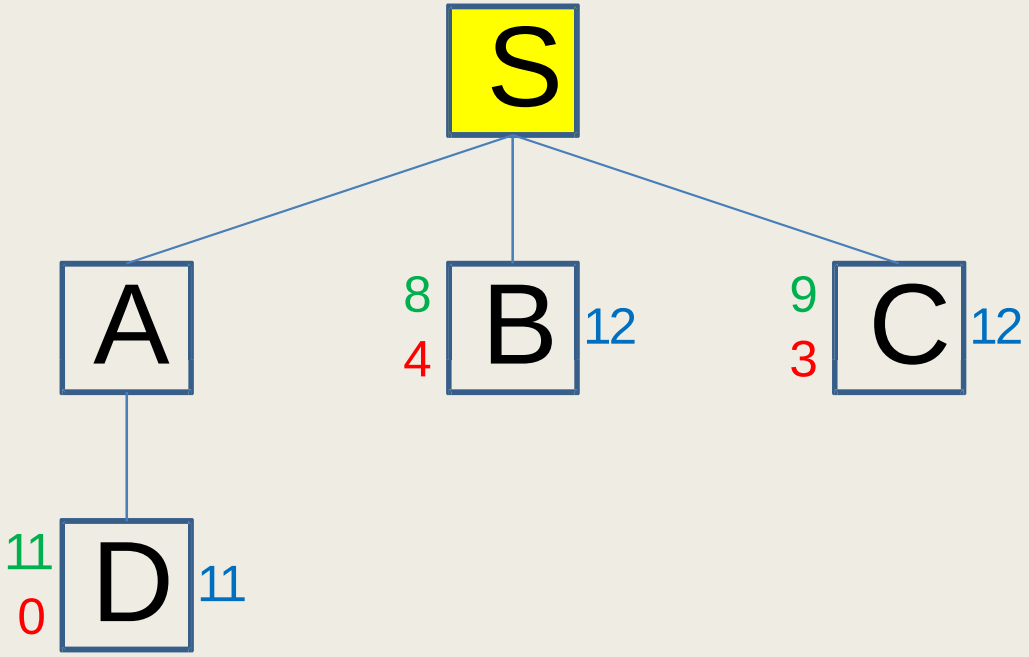


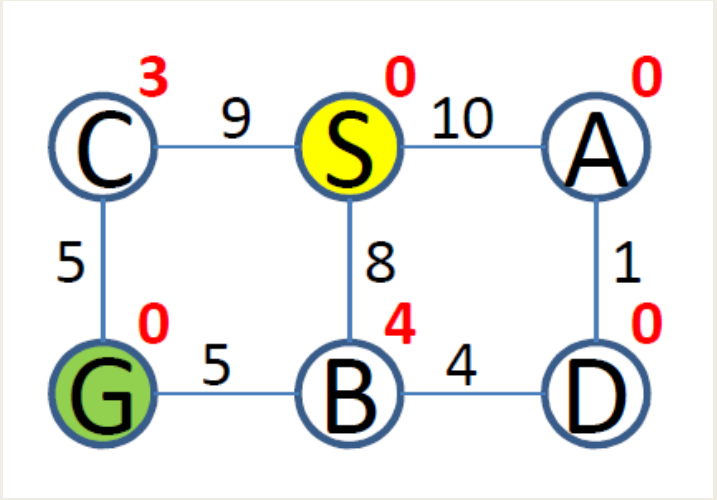


f-bound =12

f-new =∞

IDA* Search

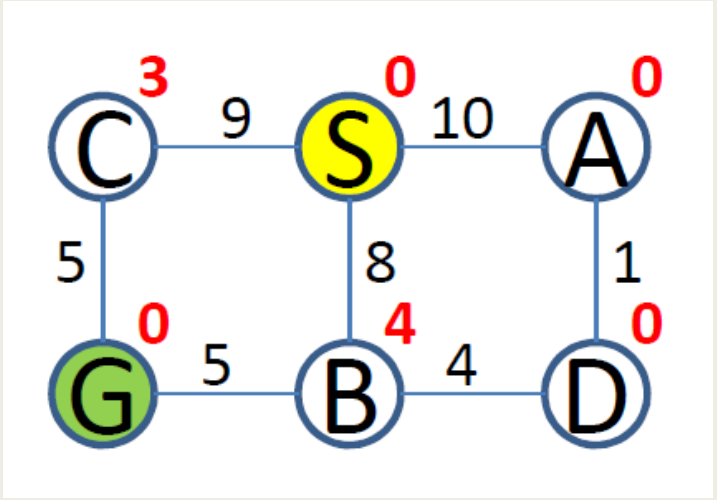


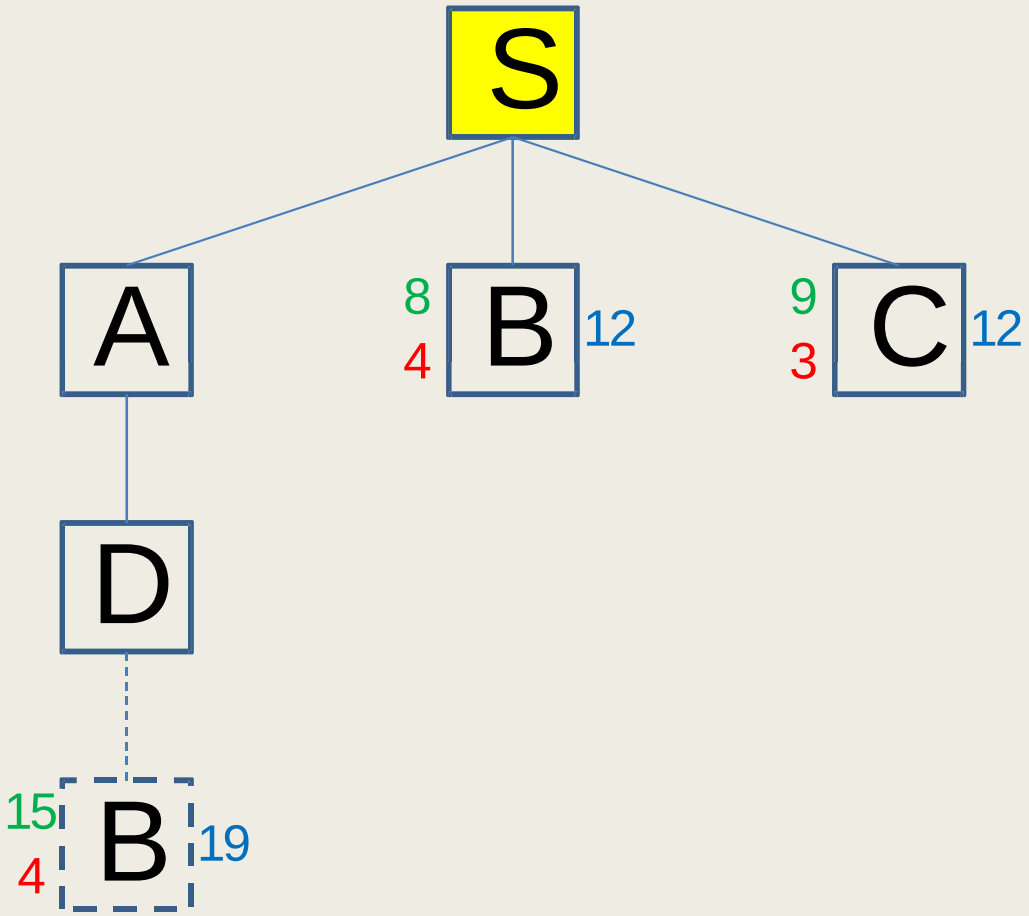


f-bound =12

f-new =∞

IDA* Search

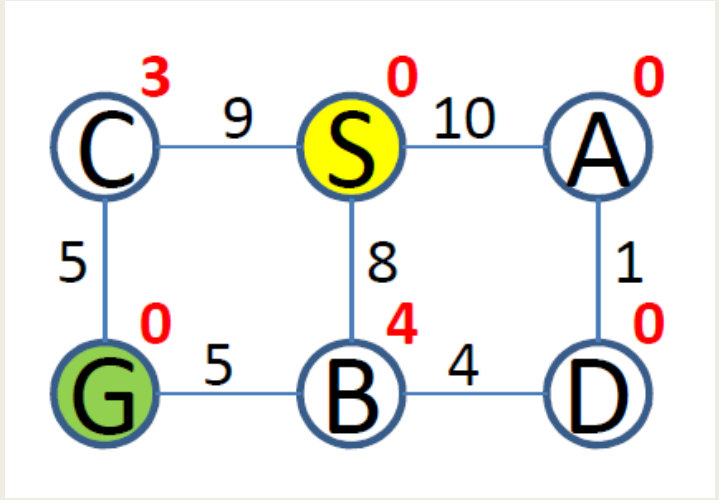




f-bound = 12

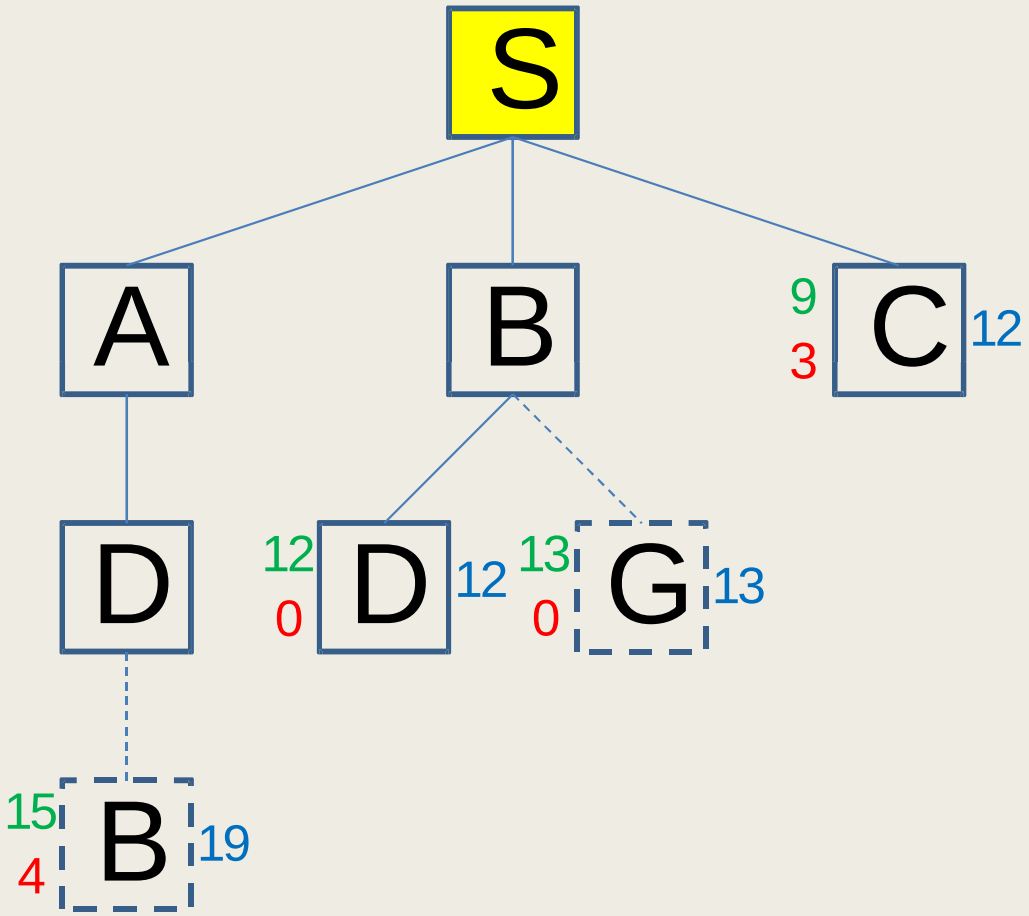
f-new = 19

IDA* Search

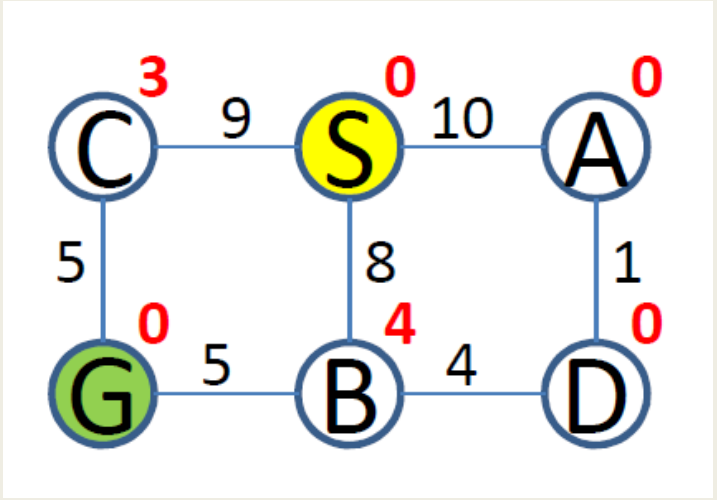


f-bound = 12

f-new = 13

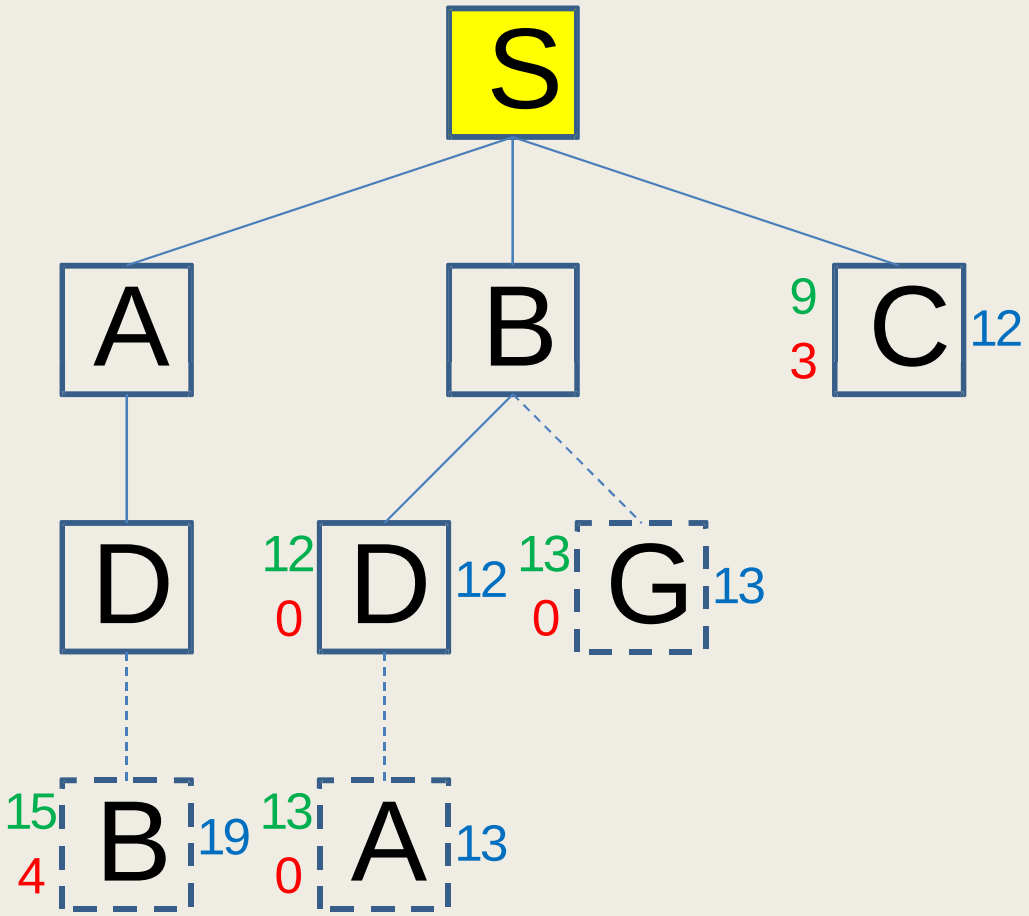


IDA* Search

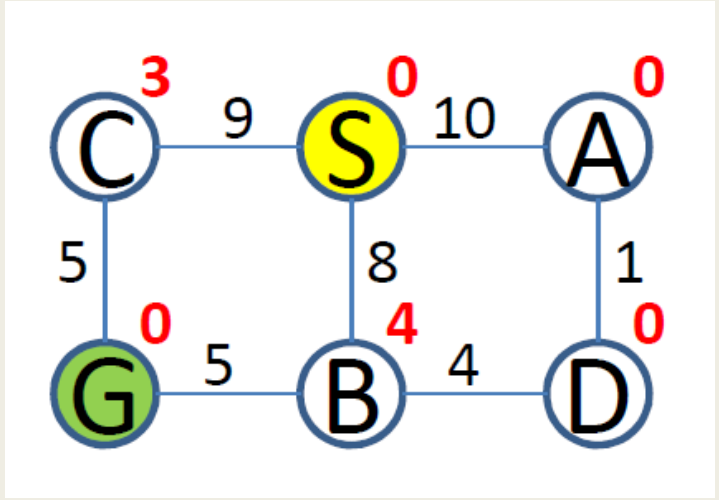


f-bound = 12

f-new = 13

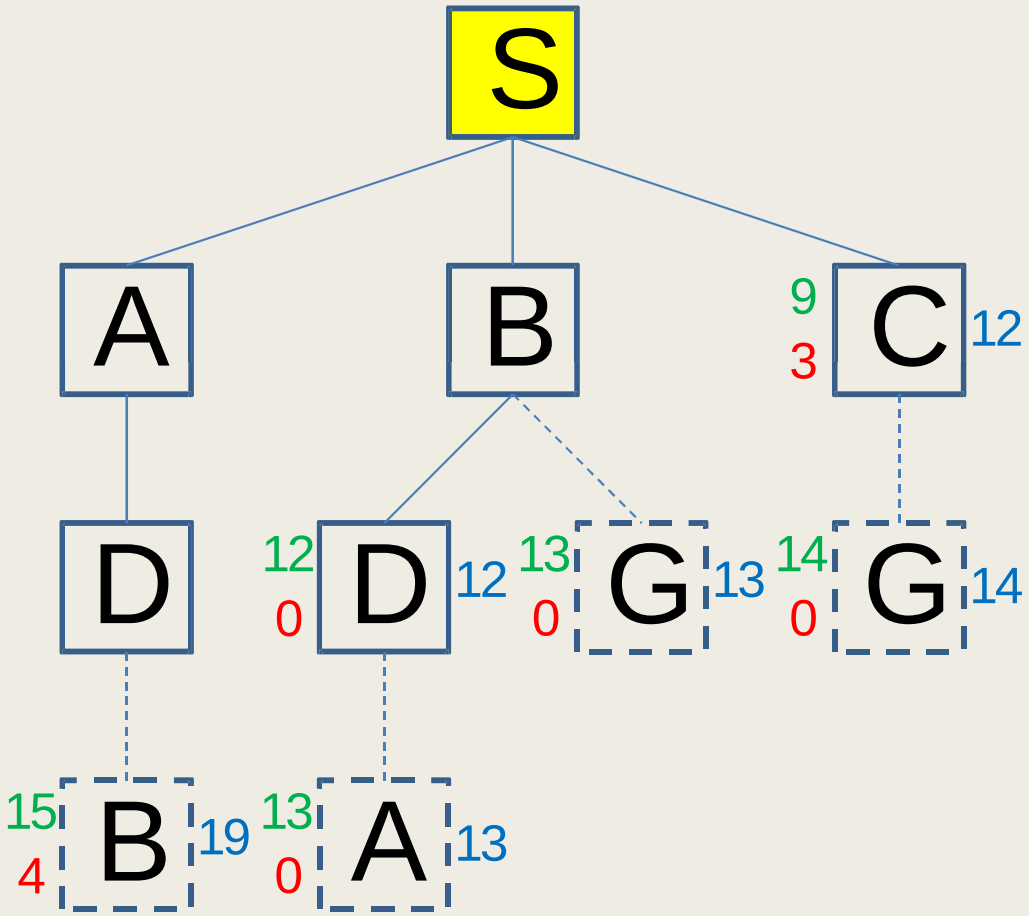


IDA* Search



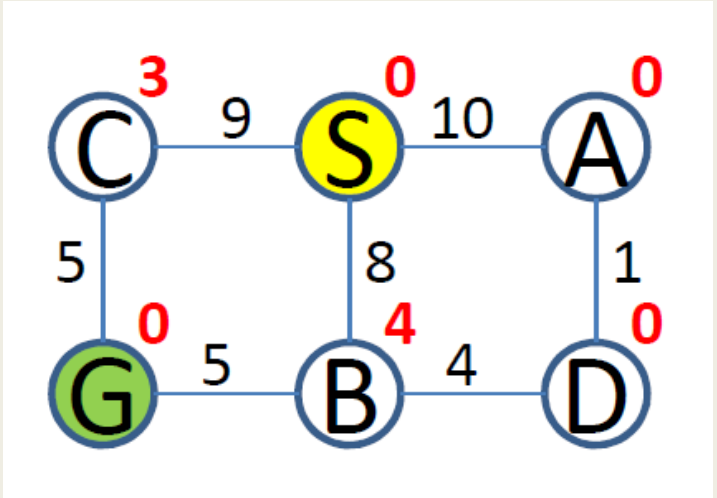
f-bound = 12

f-new = 13



IDA* Search

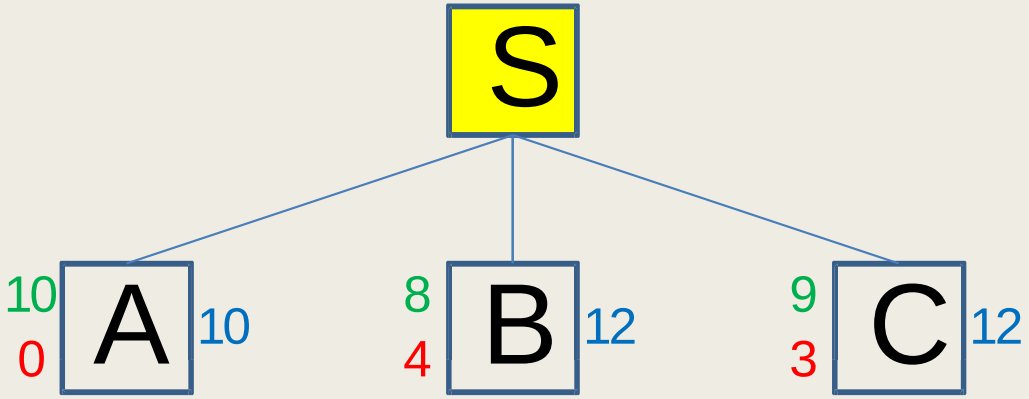


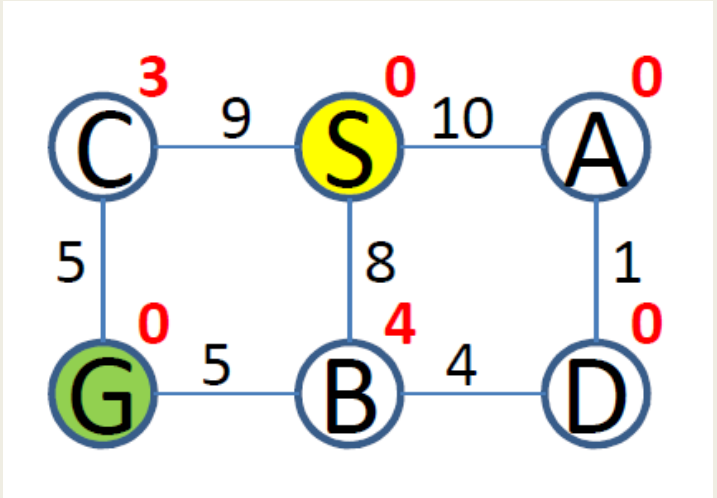


f-bound =13

f-new =∞

IDA* Search

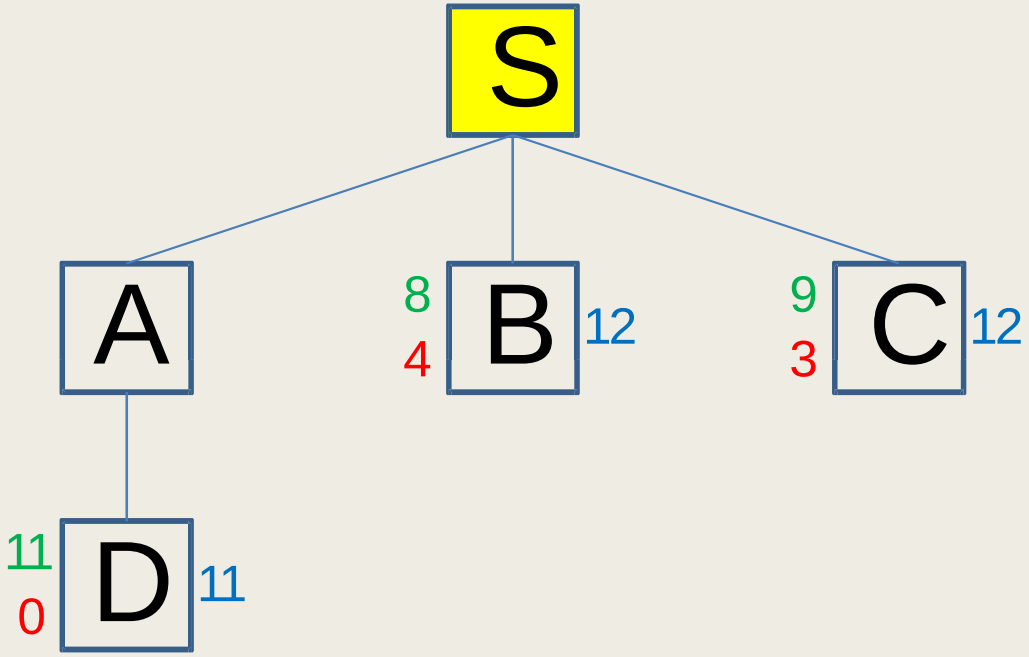


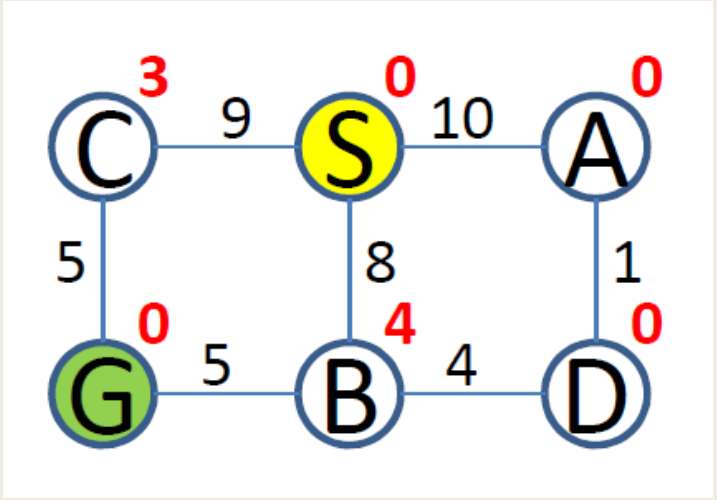


f-bound =13

f-new =∞

IDA* Search

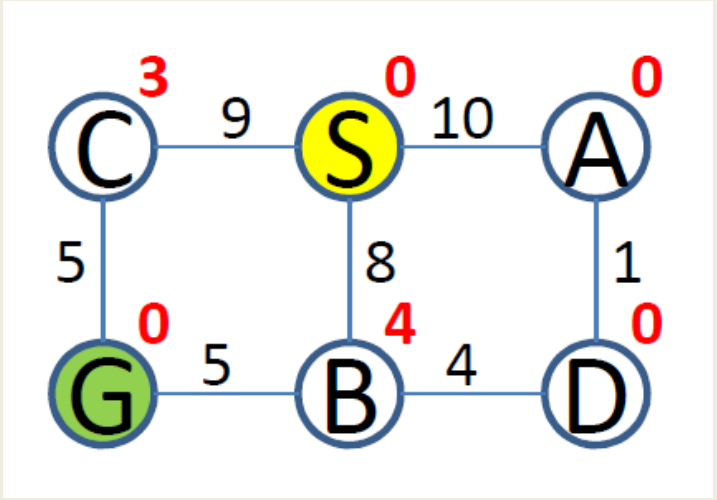




f-bound =13

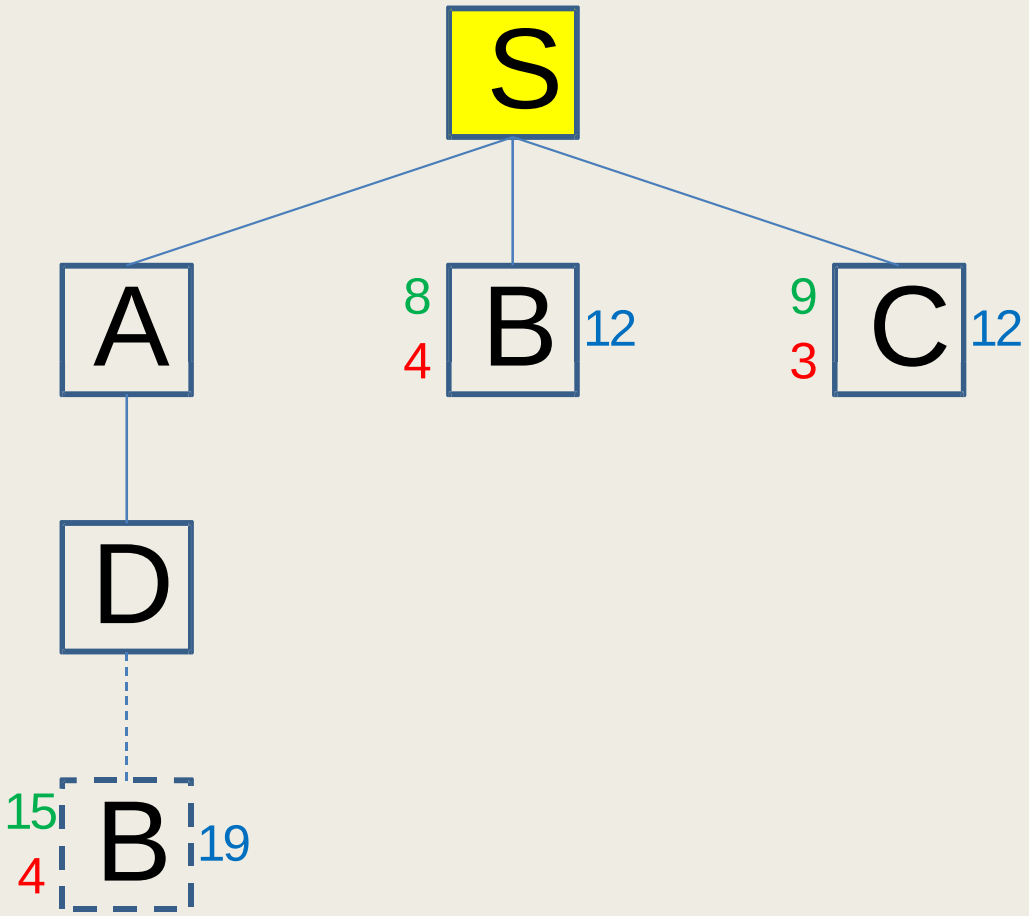
f-new =∞

IDA* Search

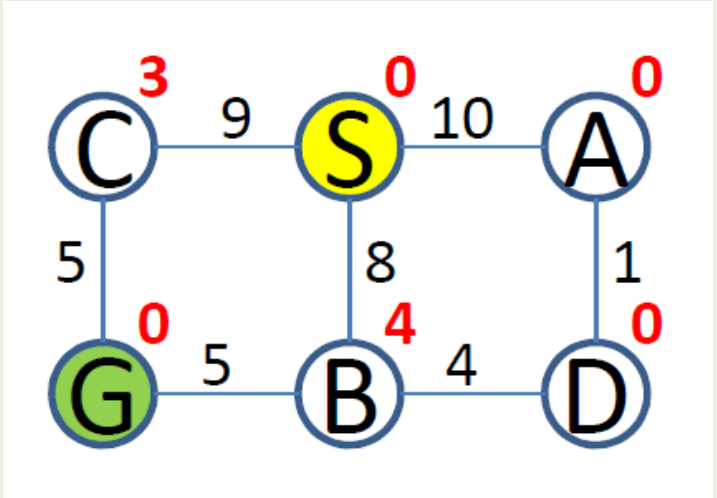


f-bound = 13

f-new = 19

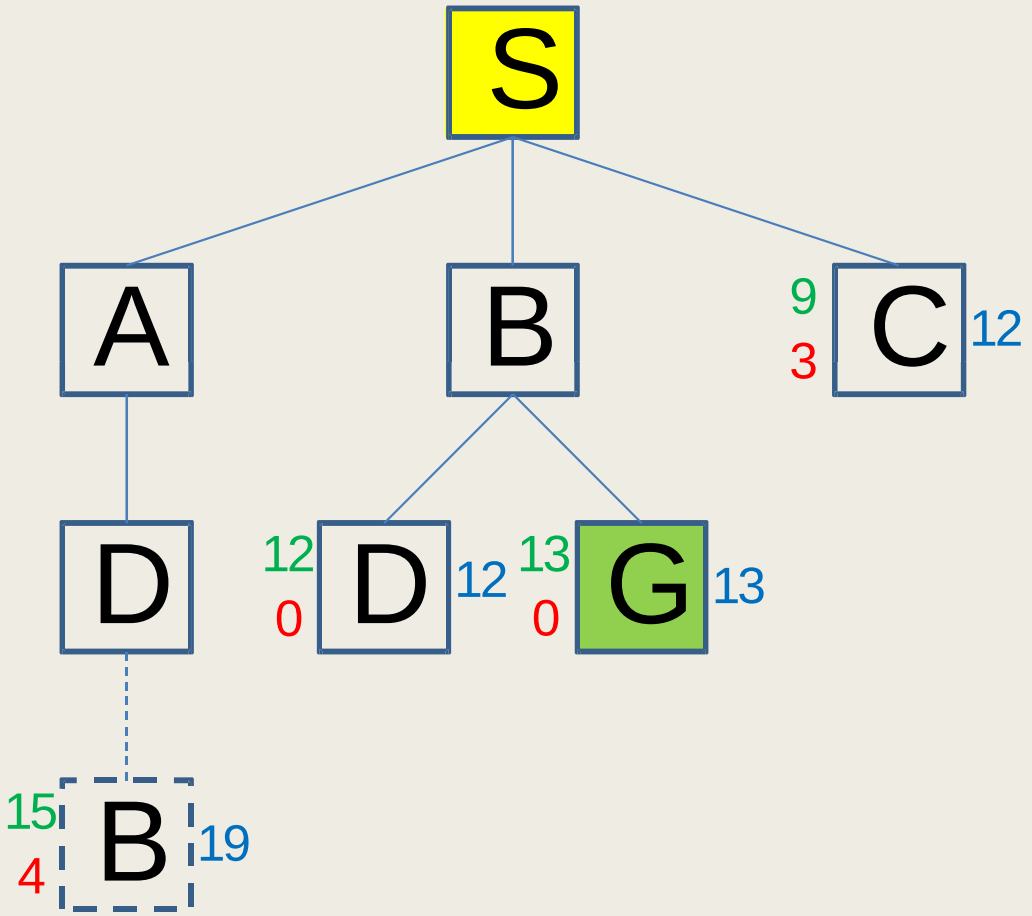


IDA* Search



f-bound = 13

f-new = 19



Properties of IDA*

- **Complete?**

  Yes

- **Optimal?**

  Yes, if the heuristic function is admissible, IDA* finds an optimal solution

- **Time**

  O (b^d), where d is the depth of the solution.

- **Space**

  O(bd), memory requirement is linear with respect to the maximum search depth.

AO* Algorithm

- AO* Algorithm basically based on problem decomposition (Breakdown problem into small pieces)
- When a problem can be divided into a set of sub problems, where each sub problem can be solved separately and a combination of these will be a solution, **AND-OR graphs** are used for representing the solution.
- The decomposition of the problem or problem reduction generates AND arcs.

AO* Algorithm

The figure shows an AND-OR graph.

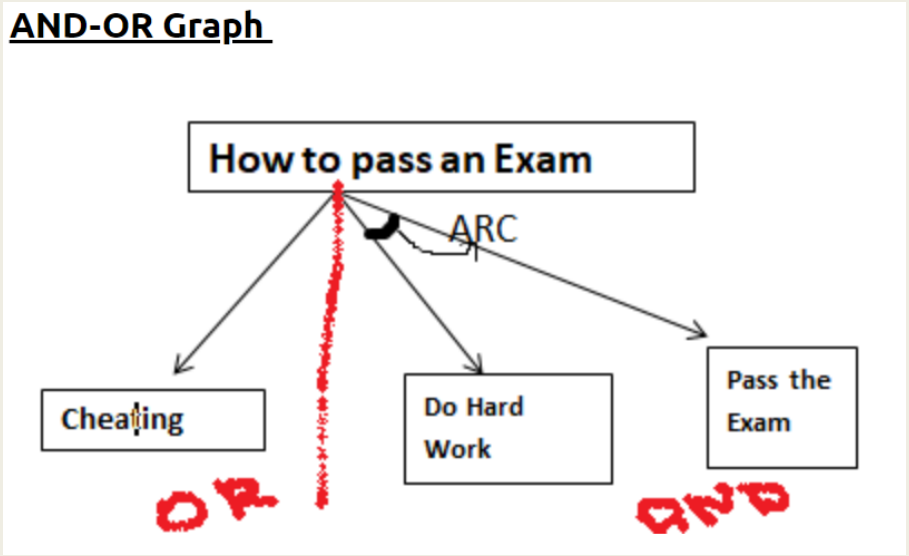
1.To pass any exam, we have two options, either cheating or hard work.

2.In this graph we are given two choices, first do cheating **or (The red line)** work hard and **(The arc)** pass.

3.When we have more than one choice and we have to pick one, we apply **OR condition** to choose one.(That's what we did here).

4.Basically the **ARC** here denote **AND condition.**

5.Here we have replicated the arc between the work hard and the pass because by doing the hard work possibility of passing an exam is more than cheating.

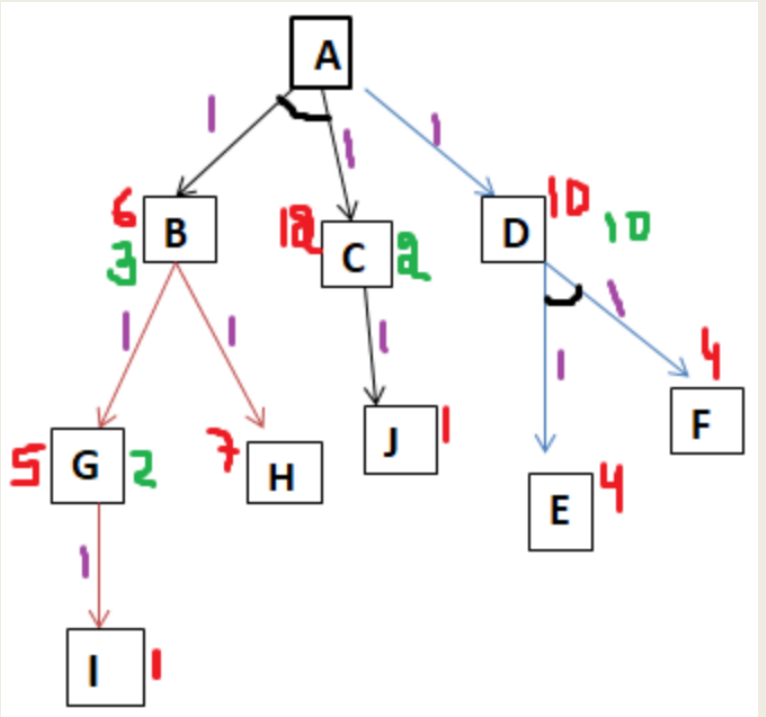


A* vs AO*

- Both are part of informed search technique and use heuristic values to solve the problem.
- A* **always** gives an **optimal solution** (shortest path with low cost) But It is not guaranteed to that **AO*** always provide **an optimal solutions.**
- **Reason:** Because AO* does not explore all the solution path once it got solution.

How AO* works?

- The algorithm always moves towards a **lower cost value.**
- Basically, We will calculate the **cost function** here **(F(n)= G (n) + H (n))**
- **H: heuristic/ estimated** value of the nodes. and **G:** actual cost or edge value (here unit value).
- Here we have taken the **edges value 1** , meaning we have to focus solely on the **heuristic value.**
- The Purple color values are **edge values (here all are same that is one).**
- The Red color values are **Heuristic values for nodes.**
- **The** Green color values are **New Heuristic values for nodes.**

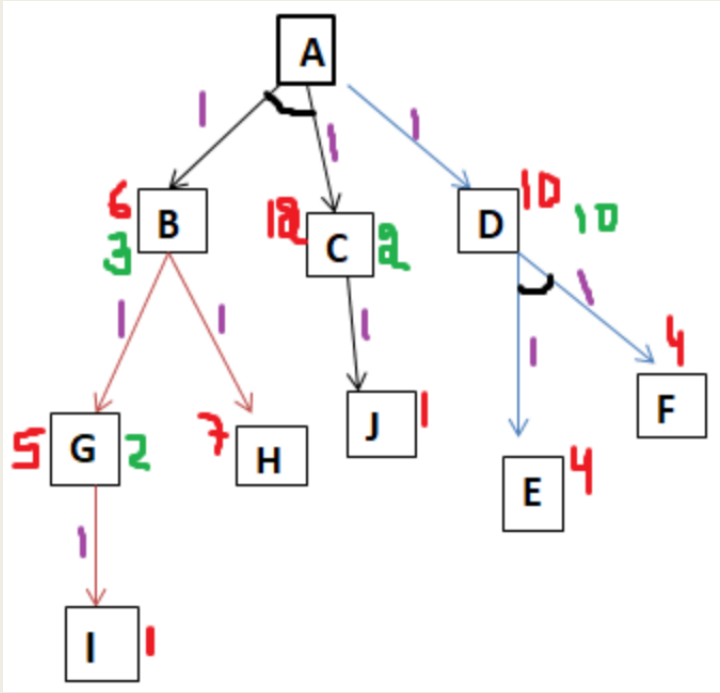


How AO* works?

- **Procedure:**

1. In the diagram we have two ways from **A to D** or **A to B-C** (because of and condition). calculate cost to select a path
2. F(A-D)= 1+10 = 11 and F(A-BC) = 1 + 1 + 6 +12 = 20
3. As we can see **F(A-D)** is less than **F(A-BC)** then the algorithm choose the path **F(A-D).**
4. From D we have one choice that is **F-E.**
5. F(A-D-FE) = 1+1+ 4 +4 =10
6. Basically **10** is the cost of reaching **FE from D.** And **Heuristic value of node D** also denote the cost of reaching **FE from D.** So, the new Heuristic value of D is 10.
7. And the Cost from A-D remain same that is **11.**

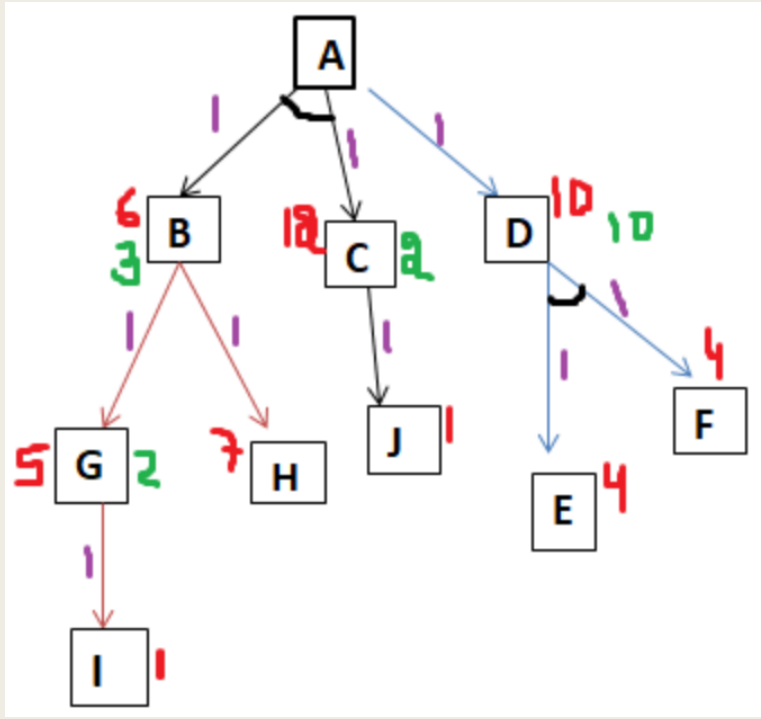
Suppose we have searched this path and we have got the **Goal State**, then we will never explore the other path. (this is what AO* says but here we are going to explore other path as well to see what happen)



How AO* works?

- **Let's Explore the other path:**

1. In the above diagram we have two ways from **A to D** or **A to B-C** (because of and condition). calculate cost to select a path
2. **F(A-D)= 1+10 = 11** and **F(A-BC) = 1 + 1 + 6 +12 = 20**
3. As we know the cost is more of **F(A-BC)** but let's take a look
4. Now from B we have two path G and H , let's calculate the cost
5. **F(B-G)= 5+1 =6** and **F(B-H)= 7 + 1 = 8**
6. So, cost from **F(B-H)** is more than **F(B-G)** we will take the path B-G.
7. The Heuristic value from **G to I is 1 but let's calculate the cost form G to I.**
8. **F(G-I) = 1 +1 = 2.** which is less than **Heuristic value 5.** So, the new **Heuristic value form G to I is 2.**



How AO* works?

9. If it is a new value, then the cost from **G to B** must also have changed. Let's see the new **cost form (B to G)**

10. F(B-G)= 1+2 =3 . Mean the **New Heuristic value of B is 3.**

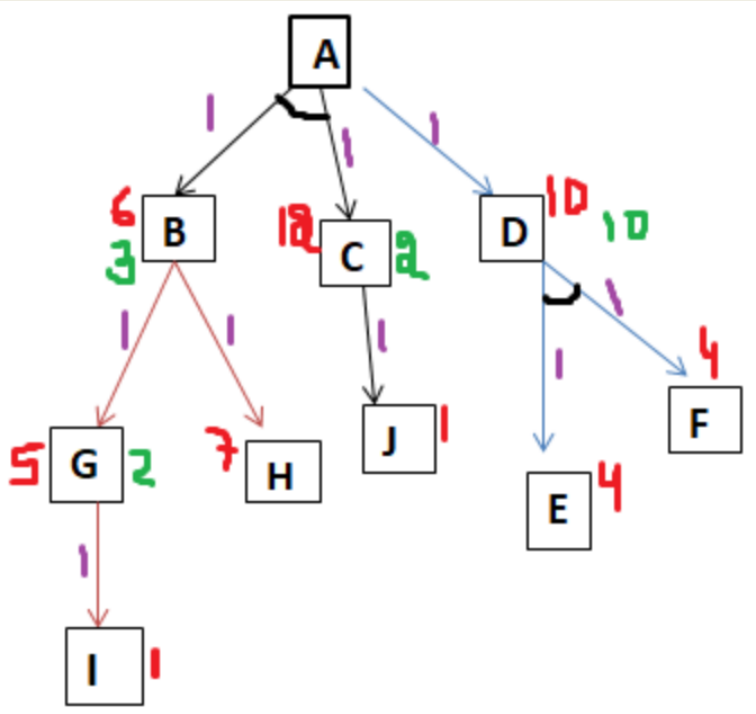
11. But A is associated with both B and C .

12. As we can see from the diagram **C only have one choice or one node to explore that is J.** The Heuristic value of C is 12.

13. Cost form C to **J= F(C-J) = 1+1= 2** Which is less than Heuristic value.

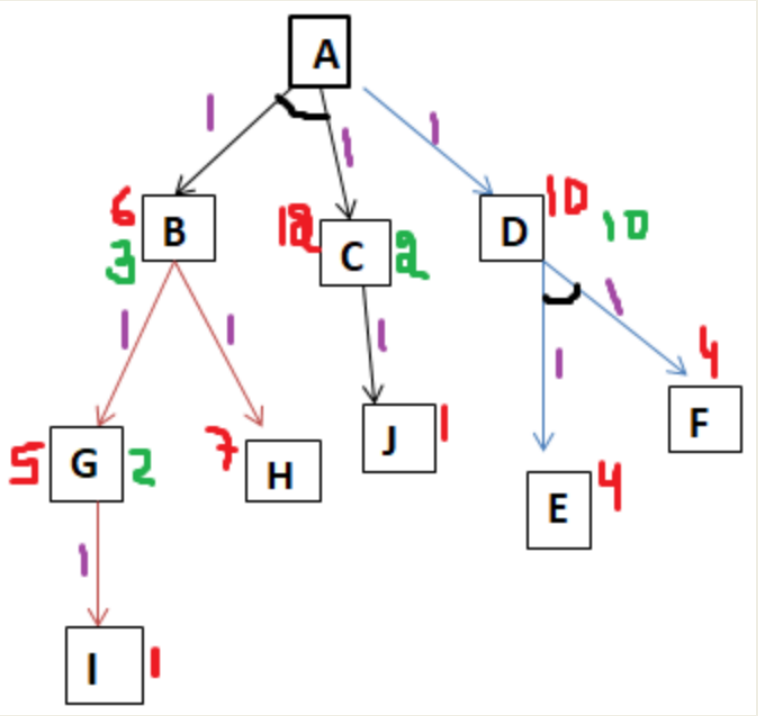
14. Now the New Heuristic value of C is 2.

15. And the New Cost from A- BC that is F(A-BC) = 1+1+2+3 = 7 which is less than F(A-D)=11.



How AO* works?

- In this case Choosing path A-BC is more cost effective and good than that of A-D.
- But this will only happen when the algorithm explores this path as well. But according to the algorithm, algorithm will not accelerate this path **(here we have just did it to see how the other path can also be correct).**
- But it is not the case in all the cases that it will happen in some cases that the algorithm will get optimal solution.



Generate-and-Test

Algorithm:

1. Generate a possible solution.
2. Test to see if this is actually a solution.
3. Quit if a solution has been found.

   Otherwise, return to step 1.

Hill Climbing

- In some problems, the path to goal is irrelevant. The goal state itself is solution.
- State space = a set of complete configuration.
- In such cases, we can use Hill Climbing algorithm.

  - Keep a single “current” state, try to improve it.
- Basic idea is to always head towards a state which is better than the current one.
- Generate-and-test + direction to move.
- Heuristic function to estimate how close a given state is to a goal state.
- Considers all the moves from the current state. Pick the best of the bunch and replace the current state with that one.
- It does not backtrack the search space, as it does not remember the previous states.

Hill Climbing

- The initial state is chosen at random!
- *The algorithm may return different solutions depending on the start state!*
- If the options do not improve the current state, remain stuck at the current state.
- It's the main difference with Greedy Search.

  - In greedy search, always move to the next (best among the options)

Hill Climbing

Let's look at the Simple Hill climbing algorithm:

1. Define the current state as an initial state
2. Loop until the goal state is achieved or no more operators can be applied on the current state:

   2.1 Apply an operation to current state and ***get a new state***

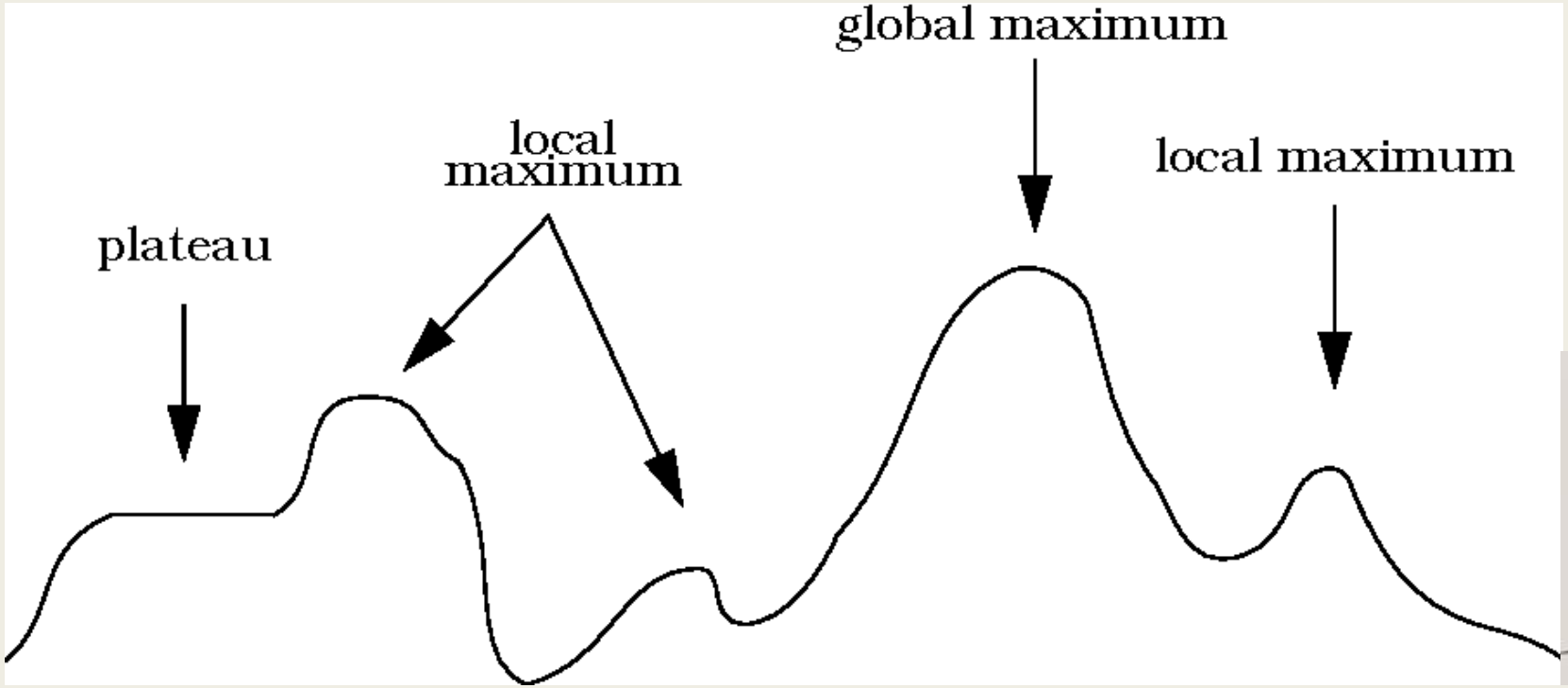
   2.2 ***Compare*** *the new state with the goal*

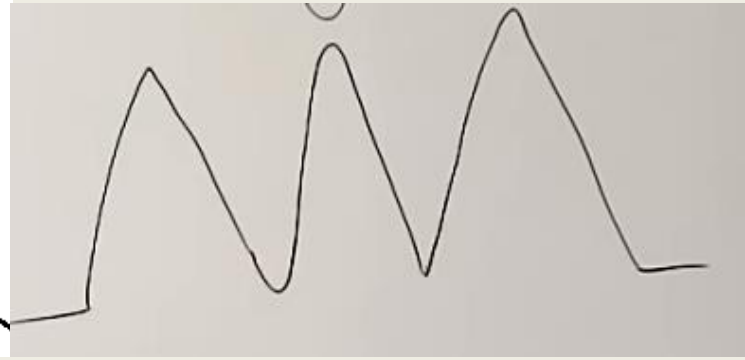
   2.3 ***Quit*** *if the goal state is achieved*

   2.4 Evaluate new state with heuristic function and ***compare it with the current state***

   2.5 ***If the newer state is closer*** *to the goal compared to current state,* ***update the current state***

Limitations of Hill Climbing





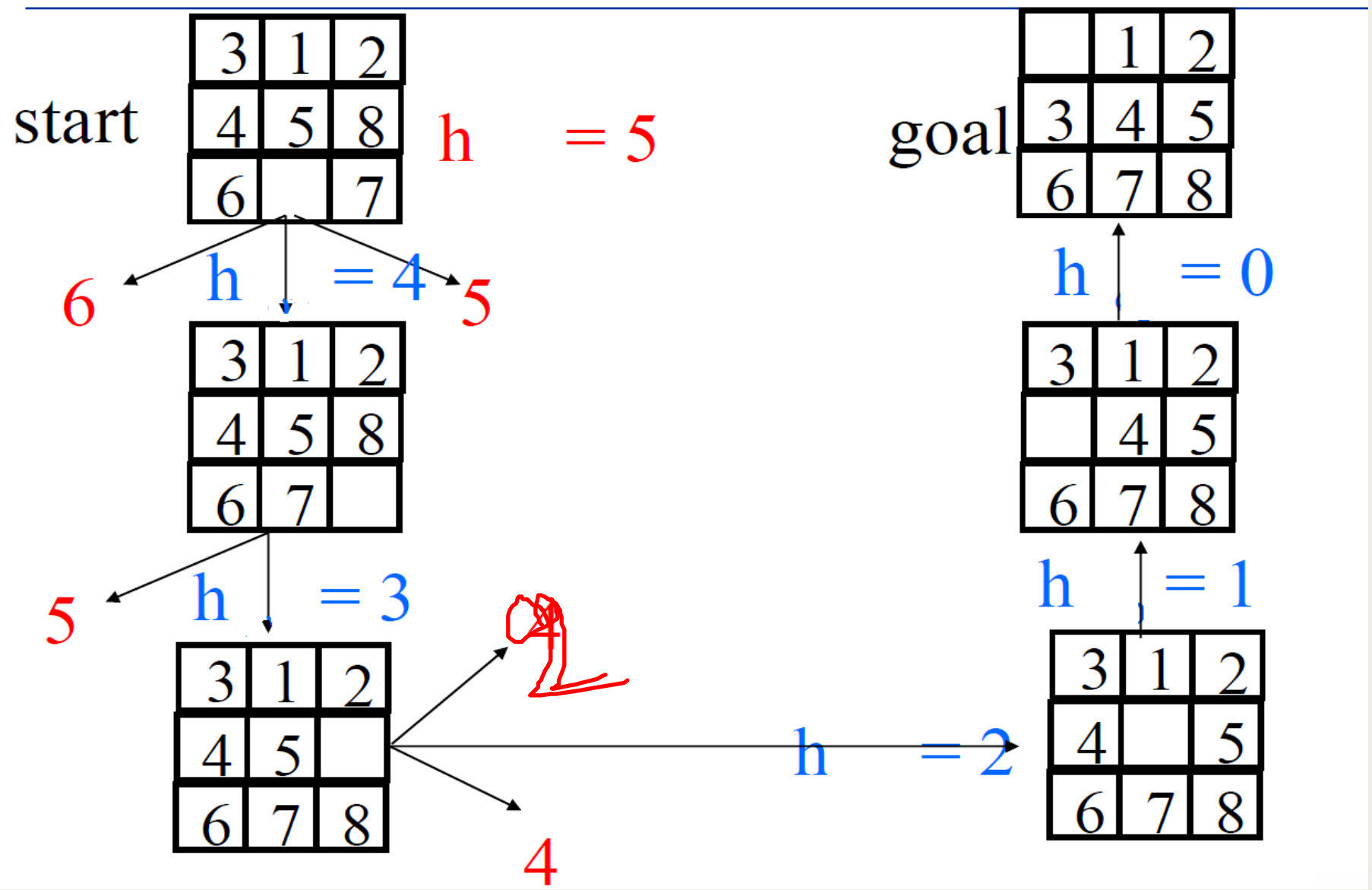
Ridge

- **Global maximum :** It is the best possible state in the state space.

Limitations of Hill Climbing

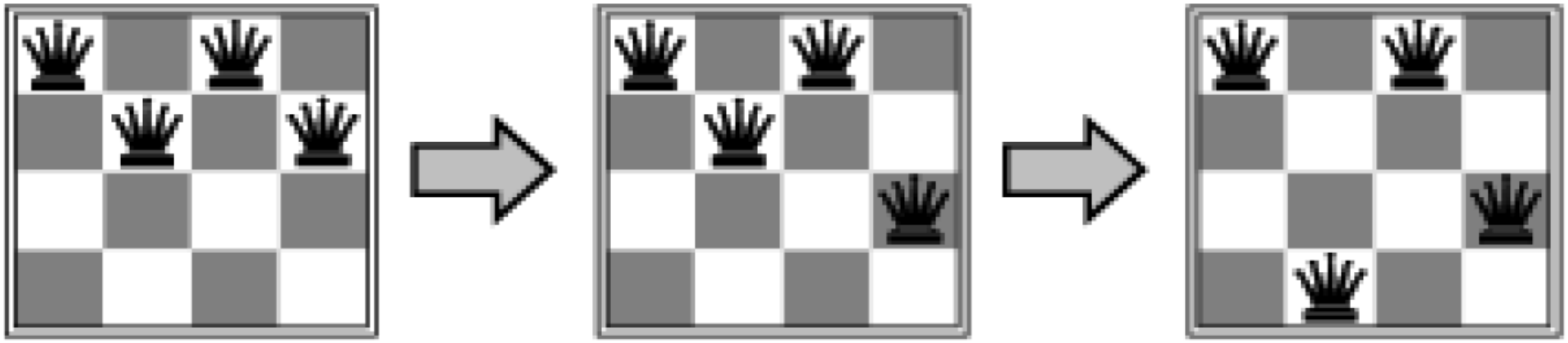
- **Local maximum :** It is a state which is better than its neighboring state however there exists a state which is better than it(global maximum). This state is better because here value of objective function is higher than its neighbors.
- **Plateua/flat local maximum :** It is a flat region of state space where neighboring states have the same value. So these are the flat points where all successor values are same.
- **Ridge :** It is region which is higher than its neighbors but itself has a slope. It is a special kind of local maximum. So it's a situation where all successors value will be same within intervals.

Hill climbing example I (*8-puzzle*)



Hill climbing example II (*n-queen*)

- ***n*-queens problem: Put *n* queens on an *n* × *n* board with no two queens on the same row, column, or diagonal**
- ***Good heuristic:* *h* = number of pairs of queens that are attacking each other**



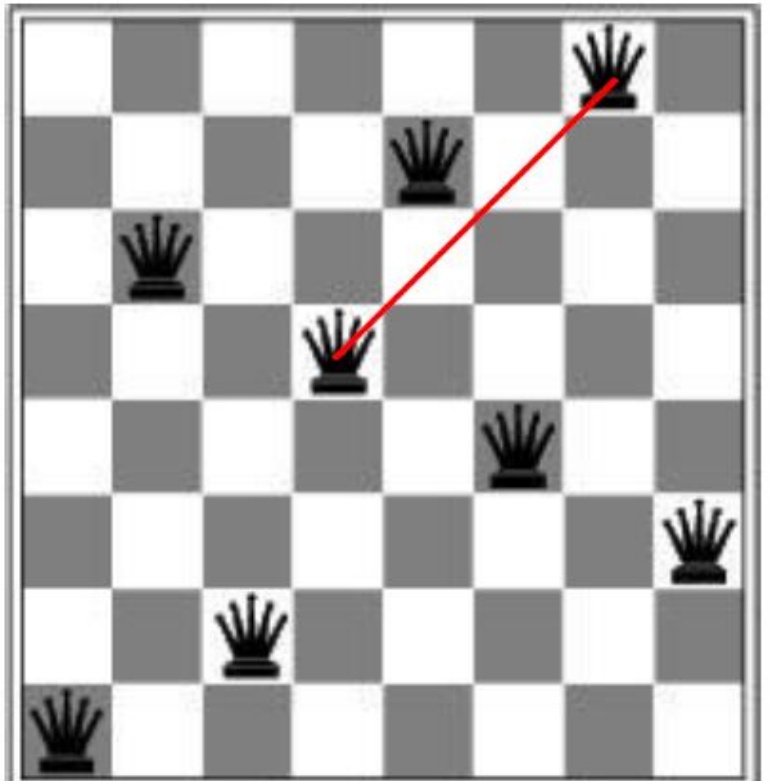
h=5 *h=3* *h=1*

(for illustration)

Hill climbing example II (*n-queen*)



A state with h=17 and the h-value for each possible successor



A local minimum of h in the 8-queens state space (h=1).

h = number of pairs of queens that are attacking each other